

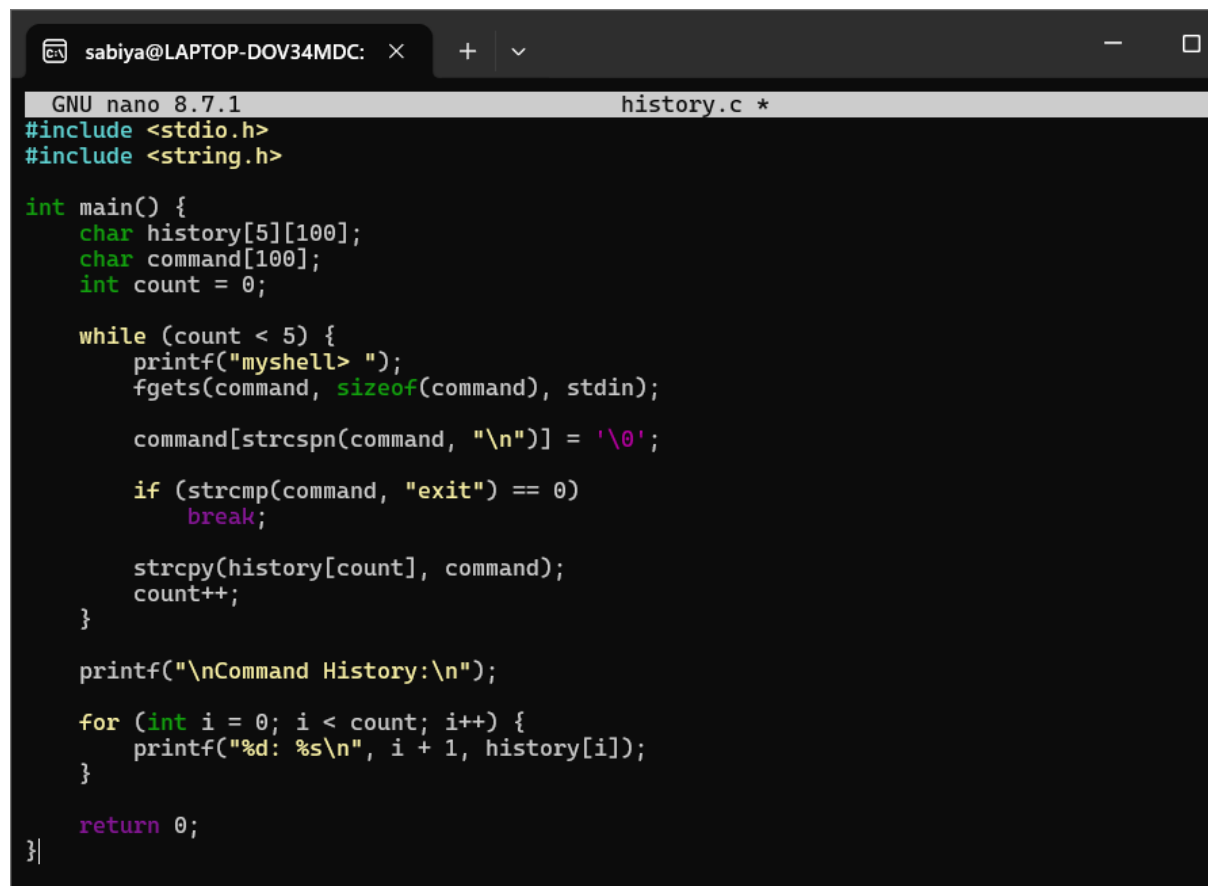
OSSP SKILL

shaik Sabiya Thabassum(2520030502)

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

(a)Code



```
GNU nano 8.7.1 history.c *
#include <stdio.h>
#include <string.h>

int main() {
    char history[5][100];
    char command[100];
    int count = 0;

    while (count < 5) {
        printf("myshell> ");
        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

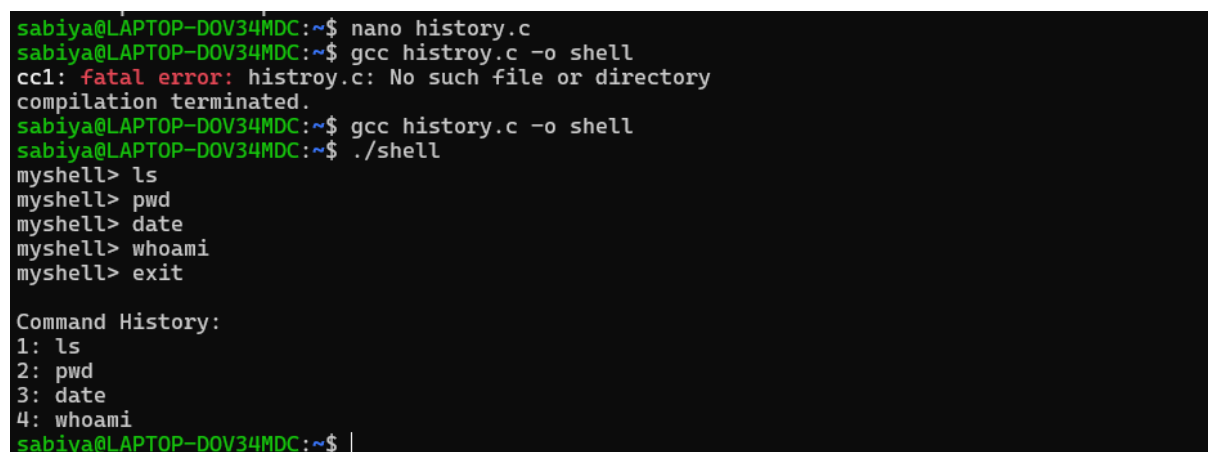
        strcpy(history[count], command);
        count++;
    }

    printf("\nCommand History:\n");

    for (int i = 0; i < count; i++) {
        printf("%d: %s\n", i + 1, history[i]);
    }

    return 0;
}
```

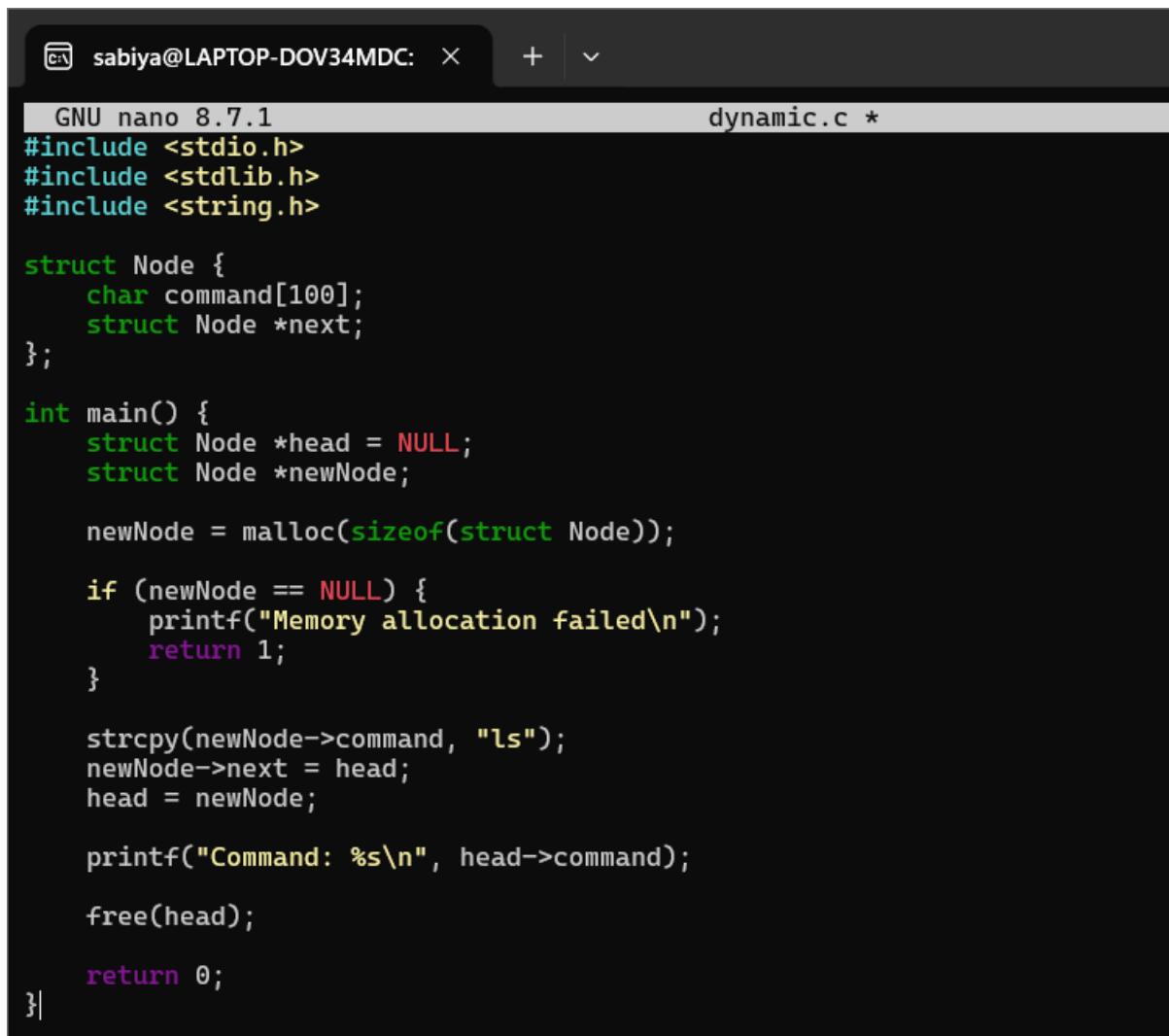
OUTPUT SCREENSHOT:



```
sabiya@LAPTOP-DOV34MDC:~$ nano history.c
sabiya@LAPTOP-DOV34MDC:~$ gcc history.c -o shell
cc1: fatal error: history.c: No such file or directory
compilation terminated.
sabiya@LAPTOP-DOV34MDC:~$ gcc history.c -o shell
sabiya@LAPTOP-DOV34MDC:~$ ./shell
myshell> ls
myshell> pwd
myshell> date
myshell> whoami
myshell> exit

Command History:
1: ls
2: pwd
3: date
4: whoami
sabiya@LAPTOP-DOV34MDC:~$ |
```

(b)Code:

A screenshot of a terminal window with a dark background. The title bar shows 'sabiya@LAPTOP-DOV34MDC:'. The terminal displays the GNU nano 8.7.1 editor editing a file named 'dynamic.c'. The code is a C program that defines a linked list structure. It includes headers for stdio, stdlib, and string. A 'Node' struct is defined with a 'command' array of size 100 and a pointer to the next 'Node'. The 'main' function initializes a 'head' pointer to NULL and a 'newNode' pointer. It then allocates memory for 'newNode' using 'malloc'. A check is made to see if 'newNode' is NULL, and if so, it prints an error message and returns 1. Otherwise, it copies the string 'ls' into the 'command' field of 'newNode', sets 'newNode->next' to 'head', updates 'head' to 'newNode', prints the command, and finally frees the memory of the original 'head' before returning 0.

```
GNU nano 8.7.1 dynamic.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char command[100];
    struct Node *next;
};

int main() {
    struct Node *head = NULL;
    struct Node *newNode;

    newNode = malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

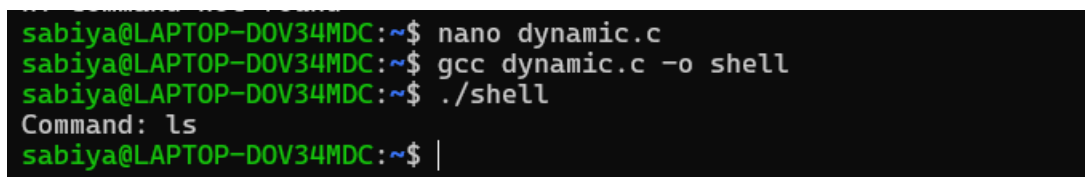
    strcpy(newNode->command, "ls");
    newNode->next = head;
    head = newNode;

    printf("Command: %s\n", head->command);

    free(head);

    return 0;
}
```

OUTPUT SCREEN SHOT:

A screenshot of a terminal window showing the execution of the program. The user runs 'nano dynamic.c', then 'gcc dynamic.c -o shell', and finally './shell'. The output shows 'Command: ls' followed by a prompt.

```
sabiya@LAPTOP-DOV34MDC:~$ nano dynamic.c
sabiya@LAPTOP-DOV34MDC:~$ gcc dynamic.c -o shell
sabiya@LAPTOP-DOV34MDC:~$ ./shell
Command: ls
sabiya@LAPTOP-DOV34MDC:~$ |
```